SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I (P.Veeraswami / 192311253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Veeraswami

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS

CO2_AT3 Error Analysis Task: Software Testing (CSA37)

Where a question does not give concrete data, a realistic example is assumed and stated, so that test cases and values can be shown.

Question 1: Equivalence Partitioning and BVA in an Online Shopping System

Assumed scenario: The "Quantity" field of a product accepts whole numbers from **1 to 10** per order. The tester's flawed design uses the classes **A: 0-5**, **B: 6-15** and **C: above 15**, and picks the test values 3, 8 and 20.

1. Errors identified

- **Valid and invalid data are mixed.** Class A (0-5) holds the invalid value 0 and the valid values 1-5. Class B (6-15) holds the valid values 6-10 and the invalid values 11-15. The members of a class do not behave alike, so one value cannot represent the whole class.
- **Classes follow equal-sized ranges, not expected behaviour.** The partitions were cut at 5 and 15, and neither is a rule in the specification.
- **Missing invalid classes:** negative numbers, decimals (2.5), non-numeric input ("abc"), special characters, empty input and very large numbers.
- **No boundary value analysis.** The values 0, 1, 10 and 11 were never tested. Values like 3, 8 and 20 sit in the middle of the classes.
- **Test 20 is the only invalid test.** It checks just one of the many ways a quantity can be invalid.

2. Root causes

- The tester split the input range numerically and did not derive partitions from the **requirement** (valid 1-10, everything else invalid).
- The tester did not know the basic rule of equivalence partitioning: every member of a class must be processed the same way, which means a class cannot hold both valid and invalid data.
- The tester assumed that testing "typical" values is enough, so the **off-by-one** errors that occur at the edges (`<` written for `<=`) were overlooked.
- There was no review or checklist for invalid data types, so only numeric ranges were considered.

3. Corrected test design

Class	Description	Partition	Representative	Expected result
V1	Valid	1 to 10 (integer)	5	Accepted
I1	Invalid: below range	0 and below	-3	Error message
I2	Invalid: above range	11 and above	25	Error message
I3	Invalid: non-numeric	letters / symbols	abc	Error message
I4	Invalid: decimal	non-integer	2.5	Error message
I5	Invalid: empty	null / blank	(blank)	Error message

Boundary value analysis for the valid range 1-10:

Test ID	Input	Boundary position	Expected result
BVA1	0	Just below minimum	Rejected
BVA2	1	Minimum	Accepted
BVA3	2	Just above minimum	Accepted
BVA4	9	Just below maximum	Accepted
BVA5	10	Maximum	Accepted
BVA6	11	Just above maximum	Rejected

4. Justification

- Each class now contains only inputs that the system treats in the same way, so one test per class covers the whole class. There are no wasted tests and no gaps.
- The six boundary tests find off-by-one errors, which are a very common fault in range checks. For example, with `if (qty > 0 && qty < 10)`, the value 10 is wrongly rejected, and only BVA5 detects it.
- The five invalid classes test the input validation, which prevents crashes, wrong orders and security problems. Every test can be traced to a requirement, so the coverage can be explained and measured.

Question 2: Random Testing in a Banking Application

Assumed scenario: A fund-transfer module allows a minimum of Rs. 100 and a maximum of Rs. 50,000 per transaction, and a daily limit of Rs. 1,00,000. The QA engineer entered random amounts and random accounts and claimed "high coverage".

1. Errors in strategy and coverage assumptions

- **"High coverage" was claimed without any measure.** The number of tests executed was treated as coverage. Coverage has to be measured against requirements, partitions or code, and none of these was used.
- **Random values rarely hit the critical points.** The chance that a random amount in Rs. 1-1,00,000 is exactly 50,000 or 50,001 is about 1 in 100,000.
- **No test design:** no test cases were derived from the specification, so there is no expected result (oracle) and no traceability to requirements.
- **Results cannot be reproduced.** A defect found by random input cannot be reliably re-created, and a fixed defect cannot be re-verified.
- **Combinations and states were ignored:** the daily limit, account status (frozen, dormant) and insufficient balance were not tested.
- **No risk-based priority:** high-risk features such as limits were tested as lightly as unimportant ones.

2. Root causes of the missed defects

- Reliance on **probability, not design**. Boundary defects are located at exact points, which random selection almost never reaches.
- A **lack of knowledge of black-box techniques** (equivalence partitioning, BVA, decision tables), and testing under time pressure with no test plan.
- The specification was not analysed for limits and rules, so no one knew which values were critical.
- Quantity was confused with quality: many tests do not imply many different behaviours tested.

3. Suitable black-box techniques

Technique	How it is applied to the banking case
Equivalence Partitioning	Valid: 100-50,000. Invalid: below 100, above 50,000, negative, zero, non-numeric, blank.
Boundary Value Analysis	Test 99, 100, 101, 49,999, 50,000, 50,001. Also the daily limit: total 99,999, 1,00,000, 1,00,001.
Decision Table Testing	Conditions: amount in limit? balance sufficient? account active? daily limit left? Each rule gives an expected action (approve, reject with reason).
State Transition Testing	Account states Active, Frozen, Locked, Closed; check which transactions are allowed in each state.
Cause-Effect / Use-case testing	End-to-end flows such as login, transfer, OTP failure, timeout and retry (to detect duplicate debits).

4. Justification: adequacy and reliability

- **Adequacy is measurable:** every partition, boundary and decision rule is a coverage item, so the team can say, for example, "all 14 boundary and 12 rule tests were executed".
- **Defect detection is higher:** the critical limit and boundary defects are targeted on purpose. A smaller number of well-designed tests finds more defects than many random ones.
- **Reliability:** tests are repeatable and can be used for regression testing after each fix, and the results give confidence for sign-off in a financial system. Random testing can still be added afterwards as a supplement, but not as the main strategy.

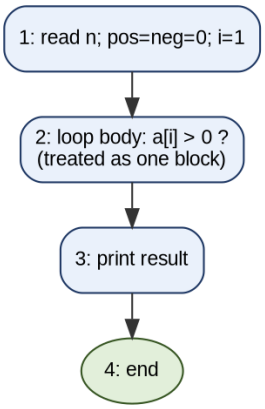
Question 3: Incomplete Control Flow Graph and Path Testing

Assumed module (counts positive and negative values in an array):

```
1  read n; pos = 0; neg = 0; i = 1
2  while (i <= n)          // loop decision
3     if (a[i] > 0)        // decision
4        pos++
     else
5        neg++
6     i++
7  if (pos > neg)          // decision
8     print "Majority positive"
   else
9     print "Not majority"
10 end
```

1. Errors in the developer's CFG and path testing

- The `while` decision (node 2) is shown as an ordinary statement, and the **back edge (6 to 2) is missing**. The CFG therefore has no loop, and the zero, one and many-iteration cases cannot be seen.
- The `if (a[i] > 0)` decision is missing, so the **true and false branches (nodes 4 and 5)** do not exist in the graph.
- The final decision `pos > neg` is missing, so nodes 8 and 9 are shown as one sequential step.
- The result is a single straight line with cyclomatic complexity V(G) = 1. The real code has V(G) = 4, so **three independent paths were never planned or tested**.

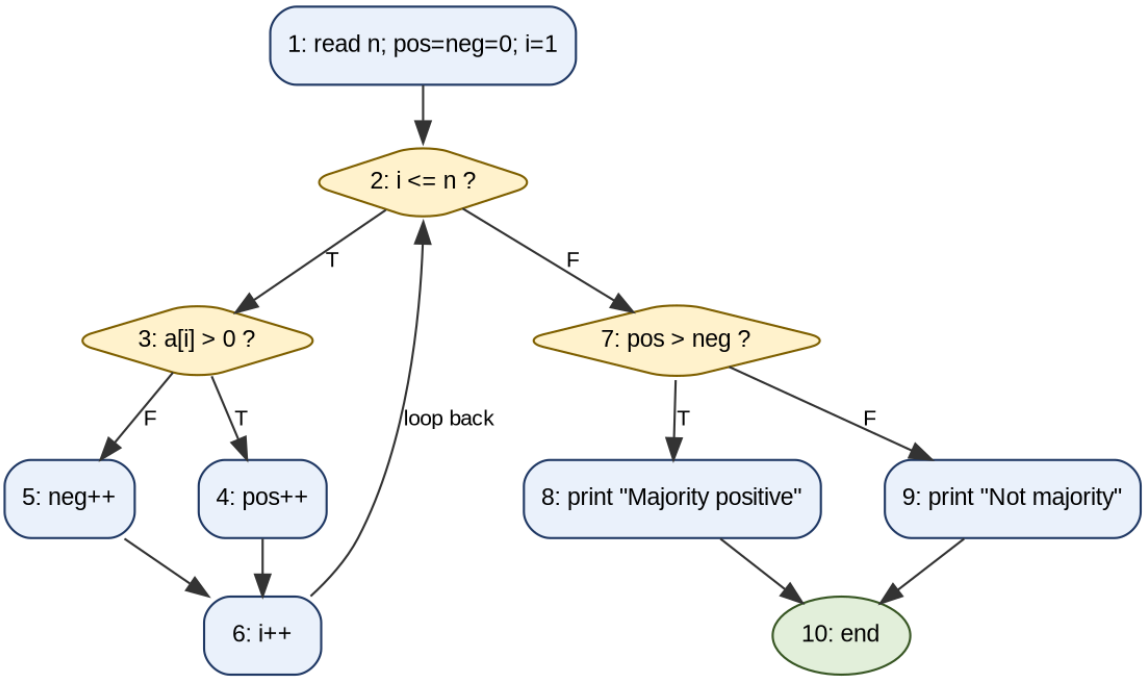


Faulty CFG (no decisions, no loop)

2. Root causes

- The developer built the graph from the **statement order** rather than from the control structures; every `if`, `while` and `else` should generate a decision node.
- The `else` parts are not written in the code (implicit), and loops were seen as a "single block", so they were left out.
- Cyclomatic complexity was never calculated as a cross-check, and the CFG was not reviewed against the code.

3. Corrected CFG



Corrected CFG (decision nodes in diamonds, loop-back edge 6 to 2)

Cyclomatic complexity: Nodes N = 10, edges E = 12, so V(G) = E - N + 2 = **4** (also = 3 predicate nodes + 1 = 4). Four independent paths are required.

Path	Node sequence	Test data	Expected output
P1	1-2-7-9-10	n = 0 (loop skipped)	Not majority
P2	1-2-3-4-6-2-7-8-10	n = 1, a = [5]	Majority positive
P3	1-2-3-5-6-2-7-9-10	n = 1, a = [-5]	Not majority
P4	1-2-3-4-6-2-3-5-6-2-7-9-10	n = 2, a = [5, -5]	Not majority (pos = neg)

4. Justification

- The four paths cover every node and every edge, including both outcomes of each of the three decisions and the loop being skipped, repeated and exited. Statement and branch coverage are therefore both 100%.
- P1 finds defects with empty input, P2 and P3 find faults in each branch (wrong comparison operator, counter updated in the wrong branch) and P4 checks the loop iteration and the tie condition `pos = neg` (a typical `>=` against `>` error).
- V(G) tells the tester the minimum number of tests required, so test adequacy is objective, and graph review shows incomplete designs before the code is run.

Question 4: 100% Statement Coverage but Logical Errors Remain

Assumed module (the rule says a discount applies for orders of Rs. 1000 or more, but the code has a logical error):

```
1  discount = 0
2  if (amount > 1000)       // BUG: should be amount >= 1000
3      discount = 10
4  total = amount - amount * discount / 100
```

1. Adequacy of the testing and gaps in coverage criteria

- The single test **amount = 2000** executes lines 1, 2, 3 and 4, which is 100% statement coverage, yet the test passes while the bug is still there.
- The **false outcome** of the `if` was never executed (the implicit else, where the condition is not met).

- No boundary (amount = 1000) was tested, so `>` instead of `>=` is not found.
- Compound conditions, loops and combinations were not examined: coverage criteria beyond statements (branch, condition, path) were absent.

The testing was therefore **inadequate**: it was adequate for the chosen metric but not for the goal of finding errors.

2. Why statement coverage alone was insufficient

- It only shows that each line **executed**, not that each line was **verified** under all decision outcomes. An `if` without an `else` has two outcomes, but only one needs to be run to cover its statements.
- It is the **weakest** structural criterion. It cannot see missing code (no else, missing check), wrong operators or wrong constants, or interactions between conditions.
- The 100% figure gave false confidence, so testing stopped early. The tests were chosen to raise the number, not to challenge the logic.

3. Recommended white-box strategies

Test	amount	Decision (amount > 1000)	Expected total	Actual (buggy) total	Result
T1	2000	True	1800	1800	Pass
T2	500	False	500	500	Pass
T3	1000	False (bug)	900	1000	**Fail: defect found**
T4	1001	True	900.9	900.9	Pass

- **Branch (decision) coverage:** T1 and T2 together cover both the true and false outcomes (100% branch coverage).
- **Condition coverage / MC/DC** for compound decisions: each atomic condition takes both values, and each is shown to affect the outcome independently.
- **Path coverage** (with basis path testing) when there are several decisions or loops.
- Add **boundary value tests** (T3) to the structural tests, since structural coverage by itself does not choose the best values.

4. Justification

- The coverage criteria form a hierarchy: statement is weaker than branch, which is weaker than condition/decision, then MC/DC, then path. Branch coverage guarantees statement coverage, but not the other way round.
- The revised tests execute both outcomes of every decision and the boundary, so hidden logical errors in conditions are exposed, as T3 proves. The metric now reflects the quality of the testing and not just execution.

Question 5: Equivalence Partitioning for an Age Field (0-100)

1. Errors in the partitioning

- **"Positive numbers" combines valid and invalid:** it includes valid ages 1-100 and invalid ages such as 101, 150 or 1000. The upper limit is not tested.
- **Zero belongs to neither class**, although 0 is a valid age (lower boundary). The two classes are not even complete.
- **Validity was ignored:** the classes are based on the sign of the number, while the specification defines validity as the range 0-100.
- The invalid inputs "non-numeric", "decimal" and "blank" are absent.
- **No BVA:** -1, 0, 1, 99, 100 and 101 were not selected.

2. Root causes

- The tester thought in terms of mathematical sign instead of the **requirement's valid and invalid sets**.
- The upper limit 100 was forgotten, as only the lower limit (negative values) was considered. The input domain was not fully analysed (type, range, format).
- Boundary analysis was regarded as optional, although equivalence partitioning gives representatives and does not locate the edges.

3. Corrected partitions

Class	Partition	Type	Sample input	Expected result
V1	0 to 100 (whole numbers)	Valid	35	Accepted
I1	Less than 0	Invalid	-5	Rejected
I2	Greater than 100	Invalid	150	Rejected
I3	Non-numeric	Invalid	abc / #	Rejected
I4	Decimal	Invalid	25.5	Rejected
I5	Blank / null	Invalid	(none)	Rejected

BVA input	-1	0	1	99	100	101
Expected	Reject	Accept	Accept	Accept	Accept	Reject

4. Justification

- The valid class is separate from the invalid classes, so a pass or fail result clearly shows whether a validity rule works.
- Both limits are tested: a defect such as `age < 100` (which rejects 100) or `age > 0` (which rejects 0) is found only by the boundary tests.
- The type, format and empty checks test **robust input validation**, which stops crashes, bad data and injection through the field. The number of tests stays small (6 classes + 6 boundaries) but the coverage is high.

Question 6: Condition Coverage for a Complex Decision

*Note: the question text in the file is cut off after `if(A && B`. The analysis below uses the decision **if (A && B)** and then extends the idea to a larger expression such as `(A && B) || C`.*

1. Errors in the developer's claim

A typical test set that gives "full condition coverage" is **(A=T, B=F)** and **(A=F, B=T)**. Each condition is true once and false once, but:

- The decision `A && B` evaluates to **false in both tests**. The "true" branch is never executed, so **decision (branch) coverage is only 50%**. Condition coverage does not imply decision coverage.
- With **short-circuit evaluation** (C, C++, Java), `B` is not evaluated at all when A is false. In test 2, B is never evaluated, so even the claim of full condition coverage is not true: B was only seen as false.
- The set does not show that each condition independently affects the outcome, so a fault such as `A || B` instead of `A && B` could go unnoticed.

2. Root causes

- Condition coverage and decision coverage were mistaken for one criterion. The first checks the atomic conditions, and the second checks the final outcome of the whole expression.
- The tester looked only at the truth values of A and B and never at the outcome of the whole expression, or at how the language evaluates operators.
- There was no truth-table analysis, and no coverage tool was used to confirm the claim.

3. Corrected tests

Test	A	B	A && B	Purpose (MC/DC)
1	T	T	T	True outcome; baseline
2	T	F	F	Changing B alone flips the outcome (shows B's effect)
3	F	T	F	Changing A alone flips the outcome (shows A's effect)

These n + 1 = 3 tests give **condition coverage, decision coverage and MC/DC**. For a larger expression like `(A && B) || C` the minimum MC/DC set is also n + 1 = 4 tests, for example (T,T,F), (T,F,F), (F,T,F) and (T,F,T). With short-circuit evaluation the tests are chosen so that every condition is actually evaluated.

4. Justification

- Tests 1-3 execute both outcomes of the decision and show that every condition matters, so errors in operators, a missing or an extra condition and wrong negation are found.
- MC/DC needs only n + 1 tests, against 2^n for exhaustive testing, so it is practical for complex conditions. It is also the level required for critical software (for example DO-178C Level A).

Question 7: Decision Table Testing for a Discount System

Assumed business rules: Premium Members get 10% off. A valid coupon gives an extra 5% off, but only if the order value is at least Rs. 2000. A rejected coupon shows an error message. The discounts are added (the stacking rule has to be confirmed with the business).

1. Errors in the construction of the decision table

- The table lists only some of the rules (for instance "Premium only" and "Coupon only") and **misses the combinations** Premium + Coupon (both), and Neither.
- A condition (**minimum order value for the coupon**) was left out, so the case "coupon applied but not accepted" has no rule.
- No completeness check was done: with n binary conditions there must be **2^n rules** (here 2^3 = 8) or documented merged rules (don't-care entries).
- There were no checks for conflicting or impossible rules, and no actions for the error message.

2. Why the approach is incomplete

The table was written from the examples that the team happened to think of and not by enumerating every combination of the conditions. Missing rules produce missing tests, so the stacking of Premium and Coupon (the most common real-life case) was never run and the price calculation was wrong in production.

3. Revised exhaustive decision table

Conditions / Actions	R1	R2	R3	R4	R5	R6	R7	R8
C1: Premium member?	Y	Y	Y	Y	N	N	N	N
C2: Coupon applied?	Y	Y	N	N	Y	Y	N	N
C3: Order value >= Rs. 2000?	Y	N	Y	N	Y	N	Y	N
A1: Premium discount 10%	X	X	X	X	-	-	-	-
A2: Coupon discount 5%	X	-	-	-	X	-	-	-
A3: Show "coupon invalid" message	-	X	-	-	-	X	-	-
Total discount	15%	10%	10%	10%	5%	0%	0%	0%

When C2 = N, C3 does not affect the outcome, so R3/R4 and R7/R8 can be merged into two rules with "-" (don't care), giving 6 rules. All 8 are kept here to show that the table is exhaustive. Each rule becomes at least one test case, for example R1: Premium, coupon, Rs. 2500 gives Rs. 2125; R2: Premium, coupon, Rs. 1500 gives Rs. 1350 with an error message.

4. Effectiveness in preventing production defects

- **Completeness is provable:** every combination of the conditions is covered, so no pricing scenario is left untested.
- The table gives the business and the testers a common view of the rules, so ambiguities (for instance, additive or multiplicative discounts) are found before coding.
- Tests are derived directly from the rules, so they are traceable, and the table can be reused for regression whenever a discount rule changes.

Question 8: Loop Testing Strategy

Assumed loop: `for (i = 1; i <= n; i++)` with a maximum of n = 10 iterations.

1. Errors in the tester's strategy

- Only the first iteration (n = 1) and the maximum (n = max) were tested.
- **n = 0 was not tested:** the loop being skipped entirely (for example, a program crash on empty input, an uninitialised result).
- **n = max + 1 was not tested:** loop overrun, array index out of bounds or `<=` instead of `<`.
- n = 2, a typical value, and max - 1 were also skipped, so errors that appear only on the second pass (state left over from the first iteration) and cumulative errors are not seen.
- Nested and concatenated loops were not considered.

2. Root causes

- The tester treated loops like ordinary boundaries, assuming that the first and last passes are enough, and did not consider that the **zero-iteration** and **overrun** cases behave differently.
- The off-by-one faults in loop conditions (`<` against `<=`) and the loop-initialisation logic were not thought about.
- No structured loop-testing checklist (such as Beizer's) was used.

3. Corrected approach

Simple loop (maximum n = 10):

Test	Iterations	What it checks
L1	0	Loop is skipped; no crash, correct default result
L2	1	Only one pass; initialisation and exit
L3	2	Second pass; state carried over from the first
L4	m (e.g. 5, typical)	Normal operation
L5	n - 1 (9)	One below the maximum
L6	n (10)	Maximum allowed
L7	n + 1 (11)	Overrun is rejected or handled; no out-of-bounds access

Nested loops (Beizer's method):

- Start with the **innermost** loop. Set all outer loops to their minimum values and apply the simple-loop tests (L1-L7) to the inner loop.
- Move outwards: test the next loop with the simple-loop set while the inner loops stay at **typical** values, and the outer ones at their minimum.
- Finally, set all loops to typical values, and then to their maximum values (stress test). This keeps the number of tests near the sum and not the product of the loop counts.
- **Concatenated loops:** if they are independent, test each as a simple loop; if the counter of one depends on the other, treat them as nested. Unstructured loops should be redesigned before testing.

4. Justification: path integrity

- The tests exercise each distinct path through the loop (never, once, several times, the limit and beyond), so the loop entry, the loop-back edge and the exit are all verified.
- Boundary and overrun tests detect the most common loop defects (off-by-one, infinite loops, uninitialised variables, buffer overflow), and the nested method keeps the test count manageable without leaving the interaction between loops untested.

Question 9: Error Guessing in a Safety-Critical Braking System

Assumed system: pedal sensor input is 0-100%, and the brake controller has the states Idle, Braking, ABS Active and Fault (fail-safe).

1. Errors in relying on ad-hoc testing

- **Error guessing depends on the tester's experience and intuition.** It is not systematic or repeatable, and it finds only the faults that the tester can imagine.
- **No coverage measurement or traceability:** nobody can show that all requirements, input ranges or states were tested, which safety standards (for example ISO 26262, IEC 61508) require as evidence.

- The sensor range edges (0%, 100%, out-of-range readings) and **state-dependent behaviour** (pressing the pedal in the Fault state) were not tested methodically.
- Sequences of events (pedal press, wheel slip, release, sensor failure) were not covered.

2. Impact on test adequacy

- Adequacy is **unknown and probably low**: untested partitions, boundary faults and invalid transitions can remain, and in a braking system an undetected defect can cause injury or death.
- There is no objective exit criterion, the results are not reproducible for regression, and the testing is not accepted in an audit or certification.

3. Corrected strategy: BVA + State Transition Testing

(a) Boundary value analysis for the pedal sensor (valid 0-100%):

Input (%)	**-1**	**0**	**1**	**50**	**99**	**100**	**101**
Expected	Sensor error, Fault	No braking	Light braking	Normal braking	Strong braking	Full braking	Sensor error, Fault

(b) State transition table (events: E1 pedal pressed, E2 wheel slip detected, E3 pedal released, E4 sensor fault):

Current state	**E1 Pedal pressed**	**E2 Wheel slip**	**E3 Pedal released**	**E4 Sensor fault**
Idle	Braking	Idle (ignore, log)	Idle	Fault
Braking	Braking (more force)	ABS Active	Idle	Fault
ABS Active	ABS Active	ABS Active	Idle	Fault
Fault	Fault (backup braking)	Fault	Fault	Fault (needs service reset)

- Tests are derived to cover **every valid transition** (0-switch coverage), then **sequences of two transitions** (1-switch coverage), e.g. Idle, Braking, ABS Active, Idle.
- Every **invalid or unexpected cell** is tested (wheel slip while idle, pedal pressed in the Fault state) to show the safe response.
- BVA values are applied inside each state, for example 99% and 100% pedal in Braking and ABS Active. Error guessing can then be added as a supplement for fault ideas such as sensor noise, power loss or very rapid pedal presses.

4. Justification: increased safety assurance

- **BVA** makes sure the limits of the sensor and the thresholds are tested, where faults are most likely. **State transition testing** checks the behaviour that depends on history and mode (ABS, fail-safe), which no single-input technique can show.
- Together they give measurable coverage (boundaries, states, transitions), repeatable tests and traceable evidence for the safety case, so the risk of a missed critical defect is reduced and the process meets the standards for safety-critical software.

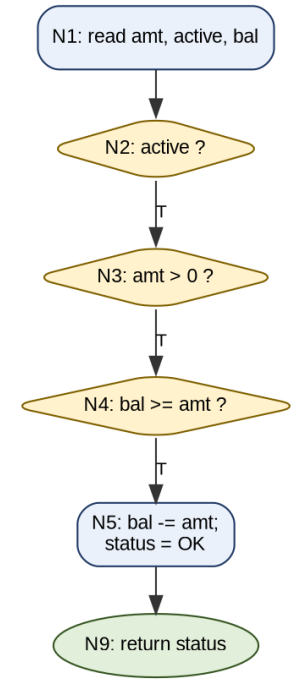
Question 10: CFG Missing the "else" Branches of Nested if Statements

Assumed module (withdrawal processing):

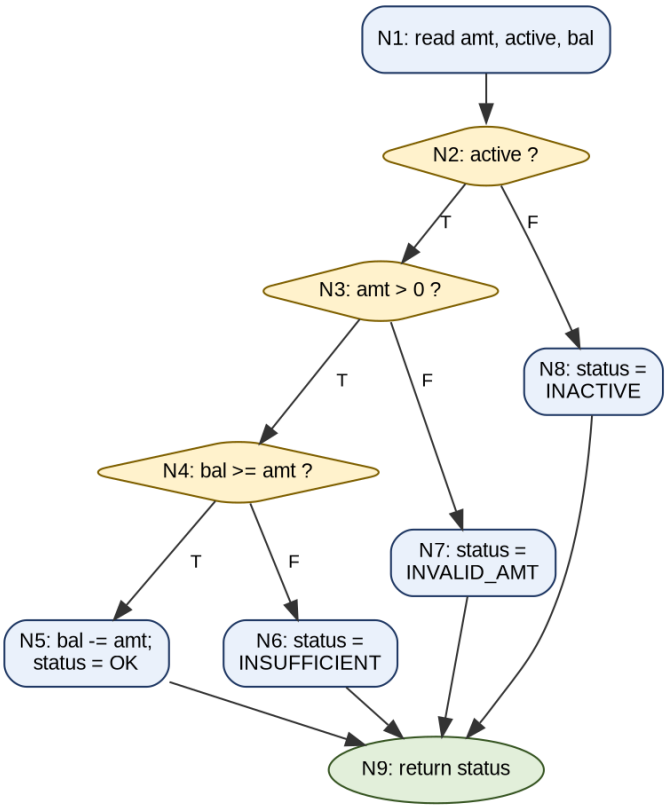
```
N1  read amt, active, bal
N2  if (active)
N3      if (amt > 0)
N4          if (bal >= amt)
N5              bal = bal - amt; status = OK
            else
N6              status = INSUFFICIENT
        else
N7          status = INVALID_AMT
    else
N8      status = INACTIVE
N9  return status
```

1. Errors in the graph construction

- Only the "true" edges were drawn (N1-N2-N3-N4-N5-N9). The three **false edges** N2 to N8, N3 to N7 and N4 to N6 are missing, and so are the nodes N6, N7 and N8.
- The three decision nodes appear as nodes with a single outgoing edge, which is not a decision. The graph has one path, V(G) = 1, while the code has V(G) = 4.
- As only the true path was drawn and tested, the report showed **100% statement coverage** for the graph, but the error-handling statements N6, N7 and N8 were never executed. **None of the false branches (0 of 3 error-path branches)** was covered, so the real branch coverage is 50% (3 of 6 branch outcomes).



Faulty CFG: only true branches



Corrected CFG: all true and false branches

2. Root causes for the missing nodes

- The **implicit or default flow** was treated as unimportant: the error paths consist of one line each, and they appear as an "else" only in the nested structure, so the tester followed the happy path.
- Each `if` was seen as a statement and not as a **decision with two outgoing edges**; the rule "every predicate node has a true and a false edge" was not applied.
- The graph was not compared with the code, and V(G) was not calculated. A tool-generated coverage report was not used to check the hand-drawn graph.

3. Corrected CFG and independent paths

Nodes N = 9, edges E = 11, predicate nodes = 3 (N2, N3, N4). **V(G) = E - N + 2 = 11 - 9 + 2 = 4 = 3 + 1**, so four independent paths are required.

Path	Node sequence	Test data (active, amt, bal)	Expected status
P1	N1-N2-N3-N4-N5-N9	true, 100, 500	OK (bal becomes 400)
P2	N1-N2-N3-N4-N6-N9	true, 100, 50	INSUFFICIENT
P3	N1-N2-N3-N7-N9	true, 0 (or -10), 500	INVALID_AMT
P4	N1-N2-N8-N9	false, 100, 500	INACTIVE

Additional boundary tests for the same paths: bal = amt (100, 100) should give OK, amt = 1 should give OK and amt = 0 should give INVALID_AMT.

4. Result: branch adequacy

- The four paths cover all 9 nodes and all 11 edges, so statement coverage and **branch coverage are both 100%**, including each error path.
- Every false outcome of N2, N3 and N4 is now exercised, so defects in the error handling (wrong status code, balance reduced in the error case, missing check) are found, which is where production failures typically occur.